



GitHub Trending Top 10

每日热门开源项目深度解读

2026-09-17

今日榜单

- 1 alibaba/open-code-review** Go
Fast, efficient, battle-tested at Alibaba's scale. Hybrid architecture code revi... 4天
- 2 cloudflare/security-audit-skill** JavaScript
A coding-agent skill for multi-phase security audits with independently verified... 2天
- 3 addyosmani/agent-skills** JavaScript
Production-grade engineering skills for AI coding agents. NEW
- 4 Tencent/BrowserSkill** TypeScript
Let AI agents use your real, logged-in browser without interrupting your work. C... NEW
- 5 alphaXiv/OpenResearch** Rust
Turn your coding agents into research agents NEW
- 6 anthropics/claude-code** TypeScript
Claude Code is an agentic coding tool that lives in your terminal, understands y... NEW
- 7 NationalSecurityAgency/ghidra** Java
Ghidra is a software reverse engineering (SRE) framework 3天
- 8 anthropics/knowledge-work-plugins** Python
Open source repository of plugins primarily intended for knowledge workers to us... 2天
- 9 Tencent/WeKnora** Go
Open-source LLM knowledge platform: turn raw documents into a queryable RAG, an ... NEW
- 10 abue-ammar/tinycast** Swift
Tinycast — a tiny, fully native macOS launcher, hotkeys, and clipboard history. 2天

#1

Go

连续 4 天

open-code-review

Fast, efficient, battle-tested at Alibaba's scale. Hybrid architecture code review tool: deterministic pipelines + LLM Agent, precise line-level comments, built-in multi-language ruleset (NPE, thread-safety, XSS, SQL injection), OpenAI & Anthropic compatible.

Stars **33,906** Forks **2,414** Today **+3,290**

<https://github.com/alibaba/open-code-review>

项目简介：Open Code Review 是阿里巴巴开源的一款 AI 代码审查 CLI 工具，脱胎于阿里内部服务数万名开发者、累计发现数百万代码缺陷的官方审查助手。它通过读取 Git diff 并调用可配置的大模型，自动生成行级精度的审查评论，解决人工代码审查效率低、遗漏多、难以规模化的问题。

核心功能：一是基于 Agent 的深度审查，模型可读取完整文件、搜索代码库、交叉参考其他变更文件，而非只看 diff 表面；二是内置多语言规则集，覆盖 NPE、线程安全、XSS、SQL 注入等典型缺陷；三是精确到行级的结构化评论输出；四是 ocr scan 支持对无 diff 的整文件/目录进行审计；五是兼容 OpenAI 与 Anthropic 接口，并支持 Claude Code、Codex、Cursor 等主流 Agent。

适用场景：适合需要规模化代码审查的研发团队，尤其是接入 CI/CD 流水线做 PR 自动审查；也适合接手陌生代码库时用 ocr scan 做整体审计。个人开发者想用 AI 辅助 review、企业想统一审查标准，都能直接受益。

技术亮点：最值得关注的是其“混合架构”——确定性流水线负责规则类检查，LLM Agent 负责需要上下文推理的审查，二者互补。官方 benchmark 显示，相比通用 Agent（如 Claude Code），它在同等模型下 Precision 和 F1 显著更高，token 消耗仅约 1/9，速度更快；代价是 Recall 略低，这是刻意用召回换精度的取舍。

上手建议：作为 CLI 工具，最快的方式是通过 npm 安装 @alibaba-group/open-code-review，配置好模型 endpoint 即可运行。想深入理解可先看官网 open-codereview.ai 和 DeepWiki 文档；贡献方面项目已获 OpenSSF Gold 认证、文档多语言齐全，建议从复现 benchmark（AACR-Bench 数据集已开源在 Hugging Face）或补充规则集入手。

#2

JavaScript

连续 2 天

security-audit-skill

A coding-agent skill for multi-phase security audits with independently verified, machine-readable findings

Stars **9,565** Forks **511** Today **+3,606**

<https://github.com/cloudflare/security-audit-skill>

项目简介：这是 Cloudflare 开源的一个"编码智能体技能"（coding-agent skill），能让 AI 编程助手变身为安全审计员，对代码库执行多阶段漏洞挖掘。它解决的核心问题是：让 AI 驱动的安全审计过程变得结构化、可验证、可追溯，而不是一次性、随意地"扫一遍"。该项目源自 Cloudflare 内部漏洞发现框架的雏形，是那套大规模系统的单仓库起点。

核心功能：一是六阶段审计流程——侦察、覆盖驱动的狩猎、候选验证、结构化输出、独立记录复核、目标中立报告；二是"覆盖账本"（coverage-ledger）机制，用 ledger 单元分配隔离的 hunter 并派 critic 找盲区，确保审计不遗漏；三是每个候选漏洞都交给全新的 verifier 去"证伪"，产出 confirmed/needs_validation/rejected 三种明确裁决；四是零依赖的 JSON 校验脚本，对 findings 和 coverage ledger 做机器可读的格式验证；五是多次运行可叠加，复用历史账本定位新缺口、复核变更源码。

适用场景：适合安全工程师、红队成员、以及希望用 AI 辅助做代码审计的开发者。典型场景是在接手一个陌生代码库时做系统性安全评估，或对已有项目做增量的、可复现的漏洞挖掘，尤其适合需要留下审计证据链、便于团队复核的合规或研究场景。

技术亮点：最值得关注的是它的"对抗式验证"设计——每个漏洞候选都交给独立的新 agent 去尝试推翻，避免单一 agent 自说自话；其次是按攻击面细分了大量狩猎类别文件（内存安全、AI/LLM 提示注入、Web 协议、供应链、云部署、数据隔离等），覆盖面非常广；再者是用 JSON Schema + 校验脚本把审计结果变成机器可读、可自动化验证的产物，而不是纯文字报告。

上手建议：用 `npx skills add` 命令安装该 skill，然后在目标代码库里对编码 agent 说"security audit this codebase"即可启动。想深入理解或贡献，建议先读 `SKILL.md` 掌握核心原则和工作流，再按需查阅各攻击类别的专项文件；若关注实现细节，可从两个 `.cjs` 校验脚本和它们的测试文件入手，这是项目里最"硬"的工程部分。

#3

JavaScript

NEW

agent-skills

Production-grade engineering skills for AI coding agents.

Stars **95,890** Forks **10,134** Today **+680**

<https://github.com/addyosmani/agent-skills>

项目简介：这是由 Google Chrome 团队知名工程师 Addy Osmani 打造的开源项目，把资深工程师的开发 workflow、质量门禁和最佳实践封装成 AI 编码代理可调用的"技能包"。它要解决的问题是：AI 代理写代码往往缺少章法，而该项目让代理在开发每个阶段都能遵循一致的工程规范。

核心功能：

- **9 个斜杠命令：** `/spec`、`/plan`、`/build`、`/test`、`/review`、`/ship` 等，完整映射"定义→规划→构建→验证→评审→发布"的开发生命周期。
- **25 个可复用技能：**如 `code-review-and-quality`（多维评审）、`test-driven-development`（强制红绿重构）、`interview-me`（逐条追问需求）等。
- **自动技能激活：**根据当前任务类型自动触发对应技能，例如设计 API 时激活接口设计技能，写 UI 时激活前端工程技能。
- **`/build auto` 自主模式：**一次批准计划后自动实现全部任务，但每个任务仍坚持测试驱动、独立提交，遇到失败或风险步骤会暂停。
- **跨代理兼容：**通过 `skills CLI` 可一键安装到 Claude Code、Cursor、Codex、Copilot、Cline 等 70+ 种代理。

适用场景：适合使用 AI 编码代理进行实际项目开发的工程师和团队，尤其是希望让 AI 产出达到"生产级"质量、而非一次性原型的人。典型场景包括：需求梳理与 PRD 生成、增量式功能开发、合并前的代码评审、Web 性能审计、代码简化重构，以及需要规范化流程的团队协作。

技术亮点：项目用 JavaScript 编写，核心思路是把"工程方法论"而非"代码片段"做成可插拔的技能单元，并通过斜杠命令与自动触发机制与代理深度集成。它强调"验证不可省略"——即便在自主模式下也坚持测试驱动和逐任务提交，体现了对 AI 生成代码质量的克制与严谨。同时借助 Vercel 的 `skills CLI` 实现跨代理标准化分发，解决了碎片化生态的兼容问题。

上手建议：最快的方式是运行 `npx skills add addyosmani/agent-skills --list` 浏览技能，再按需安装；Claude Code 用户可直接通过插件市场安装。建议先从 `/spec` 和 `/plan` 两个命令入手，体验"先规格后编码"的流程。若想贡献，可关注 `README` 中提到的单技能安装路径缺失 `references/` 目录的可移植性问题（issue #361），这是当前较明确的改进点。

#4

TypeScript

NEW

BrowserSkill

Let AI agents use your real, logged-in browser without interrupting your work. CLI + extension for browser automation across any shell-capable AI agent.

Stars **3,761** Forks **259** Today **+1,350**

<https://github.com/Tencent/BrowserSkill>

项目简介：BrowserSkill 是腾讯开源的一个浏览器自动化工具，让 AI Agent 直接复用你已登录的真实浏览器，而不是启动一个干净的测试环境。它由 bsk CLI/守护进程和浏览器扩展两部分组成，通过 shell 命令桥接各类 AI Agent 与浏览器。核心解决的是"Agent 需要操作网页，但又不该打断用户自己正在浏览的工作"这一矛盾。

核心功能：

- **复用真实登录态：**Agent 可直接操作你已登录的网站，无需额外申请测试账号或反复登录。
- **独立 Agent 窗口：**浏览器任务在单独可见的窗口中运行，用户自己的标签页不受干扰；Agent 必须显式"借用"某个标签页，用完归还。
- **跨 Agent 通用：**任何能调用 shell 的 Agent（Cursor、Claude Code、Codex、OpenClaw 等）都能通过 bsk CLI 接入，不绑定特定模型或框架。
- **人在回路（Human-in-loop）：**遇到验证码、登录、确认弹窗等只有人能完成的步骤时，Agent 可暂停并请求用户接管，处理完再继续。
- **整页截图：**支持通过快捷操作或 `bsk screenshot --full-page` 命令抓取长图。

适用场景：适合已经在用 Cursor、Claude Code 等编码 Agent 的开发者，需要让 Agent 帮忙操作后台系统、填写表单、抓取已登录页面的数据、做端到端网页测试，但又不想让 Agent 抢走自己浏览器的控制权。也适合需要人工介入的敏感操作流程（如支付确认、验证码）。

技术亮点：架构上把"CLI/守护进程"和"浏览器扩展"解耦，通过扩展与本地 daemon 通信实现控制，Agent 侧只需调用 bsk 命令，做到对 Agent 框架零锁定。显式的标签页借用/归还机制和独立 Agent Window 设计，在自动化能力与用户体验之间做了很好的平衡。针对沙箱环境还提供了 `BSK_HOME + BSK_AUTO_START=0` 的持久化守护进程方案。

上手建议：最快的方式是把 README 里那行安装指令直接丢给你的 Agent，让它按 `AGENT_INSTALL.md` 自动装好 CLI 和 skill 并引导你加载扩展；手动路线则是先装 bsk CLI、再从 Chrome/Edge 商店装扩展、最后安装 skill。想贡献代码的话，建议先跑通本地 daemon 与扩展的通信流程，理解标签页借用协议，再从文档（如 long-screenshot、sandboxed-agents）或 issue 入手。

#5

Rust

NEW

OpenResearch

Turn your coding agents into research agents

Stars **4,924** Forks **294** Today **+940**

<https://github.com/alphaXiv/OpenResearch>

项目简介：OpenResearch 是 alphaXiv 团队推出的本地优先研究工作台，目标是把 Claude Code、Codex、OpenCode、Cursor 这类通用编码智能体改造成能读文献、提假设、跑实验、产出研究成果的"科研智能体"。它解决的是当前 AI 辅助科研中"智能体会写代码但不会做研究"的断层问题，让整套科研流程可被智能体自主驱动。

核心功能：一是**并行探索**，每个研究方向分配独立的智能体会话和隔离的 git worktree，互不干扰；二是**可复现实验管理**，用 git 原生的实验树追踪变体，每次运行都绑定不可变的 commit 归档；三是**证据上下文关联**，把日志、diff、文件、结果、产物与产生它们的工作绑定在一起；四是**Autoresearch 自主循环**，能自动完成"提想法→改代码→跑实验→看证据→决定下一步"的完整闭环；五是**多后端运行**，同一份源码快照可跑在本地、SSH、Slurm、K8s、Ray、Modal 等环境。

适用场景：主要面向需要大量实验迭代的科研人员、ML 工程师和独立研究者，尤其是那些已经在用编码智能体但希望把工作流从"写代码"升级到"做研究"的人。典型场景包括文献综述与假设生成、多方向并行实验对比、远程 GPU 集群上的实验编排，以及需要严格可复现性的论文级研究。

技术亮点：用 Rust 实现 CLI 与本地服务，兼顾性能与跨平台分发；"git 原生实验树 + 不可变 commit 归档"的设计把版本控制和实验追踪统一起来，比传统实验管理工具更轻量；支持"本地优先 + 可选托管算力"的混合模式，且远程运行无需发布仓库，对隐私敏感的研究者友好。注意 README 也坦诚指出远程服务无应用层认证，需自行注意主机安全。

上手建议：最快路径是在 macOS/Linux 上执行 `curl -LsSf https://openresearch.sh/install.sh | sh` 然后 `orx up`，打开 `http://127.0.0.1:4791` 的本地面板；接着用 `orx install-skills` 把技能装进你惯用的编码智能体。想深入的话，先跑通 `orx projects`、`orx runs`、`orx exp run` 等基础命令，再尝试 `orx up --remote user@host` 连接远程 GPU。贡献方面可从文档（docs/）和 CLI 子命令的边界情况入手，项目处于快速迭代期，Issue 和文档完善是低门槛切入点。

#6

TypeScript

NEW

claude-code

Claude Code is an agentic coding tool that lives in your terminal, understands your codebase, and helps you code faster by executing routine tasks, explaining complex code, and handling git workflows - all through natural language commands.

Stars **145,819** Forks **23,524** Today **+538**

<https://github.com/anthropics/claude-code>

项目简介： Claude Code 是 Anthropic 官方推出的终端 AI 编程助手，把 Claude 大模型的能力直接嵌入到开发者的命令行工作流中。它要解决的核心问题是：让开发者用自然语言就能完成理解代码库、执行常规任务、处理 Git 操作等繁琐工作，而不必在浏览器和编辑器之间来回切换。

核心功能： - **代码库理解与问答：**能读懂整个项目结构，用自然语言解释复杂代码逻辑。 - **任务自动执行：**通过对话指令完成重构、修 bug、写测试等日常编码任务。 - **Git 工作流处理：**提交、分支、PR 等操作可直接用自然语言驱动。 - **多入口接入：**支持终端、IDE，以及在 GitHub 上 @claude 触发。 - **插件扩展机制：**仓库内置插件目录，可通过自定义命令和 agent 扩展能力。

适用场景：主要面向习惯命令行工作流的开发者、需要快速熟悉陌生代码库的工程师，以及希望把 AI 辅助编码无缝融入现有工具链的团队。典型场景包括接手遗留项目时快速理解代码、批量处理重复性修改、以及在终端里直接完成提交和 PR 流程。

技术亮点：项目用 TypeScript 编写，以 npm 包形式分发（现已推荐脚本/包管理器安装），本质是一个连接本地代码环境与 Claude 模型的 agent 层。值得关注的是它的"agentic"定位——不是简单的代码补全，而是能主动执行多步骤任务、调用工具、读写文件的智能体。同时 Anthropic 明确说明了数据收集边界和隐私保护措施（如敏感信息限期保留、不用于模型训练），这对企业用户是关键考量。

上手建议：最简路径是 MacOS/Linux 用 curl 脚本、Windows 用 PowerShell 或 WinGet 安装，然后进入项目目录运行 `claude` 即可。想深入的话，建议先读官方文档了解配置和权限模型，再研究 `plugins/` 目录学习如何自定义命令和 agent；遇到问题可用 `/bug` 命令或提 GitHub issue 反馈。

#7

Java

连续 3 天

ghidra

Ghidra is a software reverse engineering (SRE) framework

Stars **78,255** Forks **8,663** Today **+912**

<https://github.com/NationalSecurityAgency/ghidra>

项目简介：Ghidra 是由美国国家安全局（NSA）研究部门创建并维护的软件逆向工程（SRE）框架。它提供了一整套高端软件分析工具，让用户能够在 Windows、macOS 和 Linux 等平台上分析编译后的二进制代码，最初是为了解决复杂逆向工程中的规模化与团队协作问题而构建的。

核心功能：反汇编与汇编、反编译（将二进制还原为类 C 代码）、函数调用图与数据流可视化、脚本自动化（支持 Java 和 Python），以及对多种处理器指令集和可执行格式的广泛支持。此外，它既能以交互模式使用，也能以自动化批处理模式运行。

适用场景：安全研究员、恶意软件分析师、漏洞挖掘人员以及逆向工程学习者都会用到它。典型场景包括分析可疑样本、审计闭源软件的安全性、理解遗留系统，以及在 CTF 竞赛或漏洞研究中定位潜在缺陷。

技术亮点：作为 Java 编写的跨平台框架，它具备高度可扩展的插件与脚本体系，用户可自行开发扩展组件。其反编译器质量高、支持处理器架构广泛，并且 NSA 将其开源本身就是一个标志性事件——把原本机构内部的高端工具带给整个安全社区。

上手建议：普通用户只需安装 JDK 25、下载官方 release 压缩包并运行 ghidraRun 即可；想写脚本或扩展的开发者可关注 Eclipse 的 GhidraDev 插件或 VS Code 集成。若要参与核心开发，则需配置 Gradle、Python3 等构建工具，从源码执行 `gradle buildGhidra` 生成开发版，并阅读开发者指南。注意先查看安全公告，避开存在已知漏洞的版本。

#8

Python

连续 2 天

knowledge-work-plugins

Open source repository of plugins primarily intended for knowledge workers to use in Claude Cowork

Stars **24,459** Forks **2,932** Today **+287**

<https://github.com/anthropics/knowledge-work-plugins>

项目简介：这是 Anthropic 官方开源的一套 Claude 插件集合，把 Claude 从通用助手变成针对特定岗位（销售、法务、财务、数据等）的"专家"。它解决的核心问题是：让知识工作者不必每次重复解释工作方式、工具和数据来源，而是通过插件把团队的工作习惯和流程固化下来，获得更一致、更专业的输出。

核心功能：一是提供 11 个开箱即用的岗位插件（生产力、销售、客服、产品、市场、法务、财务、数据、企业搜索、生物研究、插件管理），每个都预置了该职能的技能与工作流；二是通过 MCP 连接器打通 Slack、Notion、Jira、HubSpot、Snowflake 等外部工具，让 Claude 能直接读写真实业务系统；三是提供斜杠命令（如 `/sales:call-prep`、`/data:write-query`）触发显式操作，以及自动生效的 Skills 提供领域知识；四是支持企业按自身术语、工具和流程定制插件。

适用场景：主要面向企业中的知识工作者——销售做客户调研和通话准备、法务审合同和分流 NDA、财务做对账和报表、数据团队写 SQL 和搭看板等。也适合团队负责人把公司特有的工作规范沉淀成插件，让全员获得统一的 Claude 使用体验。

技术亮点：采用统一且清晰的目录结构（`plugin.json` 清单 + `.mcp.json` 工具连接 + `commands/` 显式命令 + `skills/` 自动技能），全部基于文件、易于版本管理和二次开发；深度绑定 MCP（Model Context Protocol）标准来对接外部工具，同时兼容 Claude Cowork 和 Claude Code 两个产品。

上手建议：普通用户可直接在 Cowork 内从 `claude.com/plugins` 安装，或在 Claude Code 中通过 `claude plugin marketplace add anthropics/knowledge-work-plugins` 添加市场后安装具体插件；想贡献或定制，建议先阅读 `cowork-plugin-management` 插件和某个现成插件（如 `sales`）的目录结构，理解 `skills` 与 `commands` 的写法后再动手改造。

#9

Go

NEW

WeKnora

Open-source LLM knowledge platform: turn raw documents into a queryable RAG, an autonomous reasoning agent, and a self-maintaining Wiki.

Stars **26,067** Forks **3,539** Today **+1,123**

<https://github.com/Tencent/WeKnora>

项目简介：WeKnora 是腾讯开源的企业级 LLM 知识平台，用 Go 编写，目标是把散落的原始文档转化为可查询、可推理、可持续演进的知识资产。它解决的是企业文档"存得下但用不起来"的问题——通过 RAG、Agent 和自动 Wiki 三条路径，让静态文件变成能对话、能推理、能自我维护的活知识库。

核心功能：一是 **RAG 快速问答**，支持 PDF、Word、Excel、图片、XMind 等 10+ 格式的语义检索；二是 **ReAct Agent**，可自主编排检索、MCP 工具、沙箱执行和网络搜索来完成多步复杂任务；三是 **Wiki 模式**，由 Agent 将原始文档蒸馏成自维护、互链的 Markdown 知识库，并配交互式知识图谱、版本历史与一键回滚；此外还有跨会话长期记忆、分块编辑与修订、多源数据接入（飞书、Notion、语雀、GitLab 等）以及多工作区 RBAC 权限体系。

适用场景：面向需要私有化部署、对数据主权敏感的企业团队，典型场景包括企业内部知识库问答、客服/技术支持文档助手、研发团队的代码与文档检索、以及通过企业微信/飞书/Slack/Telegram 等 IM 渠道直接对外提供问答服务。也适合需要把知识库嵌入自有网站或通过 API 集成的开发者。

技术亮点：架构完全模块化，LLM、向量库、存储后端均可替换，支持本地和私有云部署；集成 20+ LLM 提供商（含 LiteLLM、Ollama），并内置 Langfuse 全链路可观测性与运行时任务队列看板；采用 principal 模型的 scoped API key 和每工作区独立存储后端，配合 4 层角色矩阵的 RBAC，达到企业级治理要求；沙箱支持 Docker/E2B/Cube 多后端，兼顾隔离性与灵活性。

上手建议：从官方站点 weknora.weixin.qq.com 和 README 的 Getting Started 入手，用 Docker 快速拉起本地实例体验 RAG 问答和 Wiki 模式；开发者可重点阅读 Architecture 与 Developer Guide 两节，理解模块边界后再决定是接入新的数据源、LLM 提供商还是存储后端。贡献方向建议优先考虑 README 中提到的"更多数据源接入"和文档格式解析扩展，这两块需求明确且社区价值高。

#10

Swift

连续 2 天

tinycast

Tinycast — a tiny, fully native macOS launcher, hotkeys, and clipboard history.

Stars **5,974** Forks **276** Today **+738**

<https://github.com/abue-ammar/tinycast>

项目简介：Tinycast 是一款完全原生、极致轻量的 macOS 启动器，用一个全局热键把应用启动、剪贴板历史、文件搜索等日常操作集中到一个命令面板里。它瞄准的是 Raycast、Alfred 这类工具的用户，但主打"小"——内存占用不到 100 MB、零第三方依赖、无 Electron、无遥测。

核心功能： - **启动器 + 全局/按应用热键：**模糊搜索启动任意应用、固定收藏、一键退出应用。 - **剪贴板历史：**支持文本和图片、可搜索，并能直接粘贴回原应用。 - **计算器与快捷链接：**面板内直接算数学、单位、实时汇率与加密货币换算；把 URL、搜索、文件路径变成带占位符的命令。 - **窗口管理 + 系统操作：**34 种 Rectangle 风格的分屏/移动动作，以及锁屏、睡眠、清空废纸篓等系统动作。 - **生态兼容：**能原生运行现有 Raycast 扩展（用 SwiftUI 渲染），并支持从 Raycast 导入配置。

适用场景：适合追求效率、又对性能和隐私敏感的 macOS 用户——尤其是嫌 Electron 类工具吃内存、或不想被遥测和订阅绑架的人。日常写作、编程、多窗口办公时，用热键唤起面板完成启动、粘贴、分屏、翻译等高频操作，几乎不用离开键盘。

技术亮点：用 SwiftUI + AppKit 纯原生实现，**零第三方依赖**，是它能在 100 MB 内存内跑起来的根本原因。最值得关注的是它把 Raycast 扩展以 SwiftUI 原生方式渲染运行，等于在原生框架上复刻了一套第三方插件生态，工程难度不低。要求 Swift 6.0 与 macOS 26+，也说明它紧跟苹果最新技术栈。

上手建议：普通用户走 Homebrew 安装即可（注意区分 Apple Silicon / Intel / 旧系统三种 cask，brew 会自动校验机型）。想贡献的话，先读 README 和 Discord 社区了解架构，从复现构建、修小 bug 或补文档入手；由于是 AGPL-3.0 协议，二次开发需注意开源义务。

数据来源: github.com/trending

报告日期: 2026-09-17

由 DeepSeek AI 提供智能分析 | 自动生成